CS 301
High-Performance Computing

Assignment 06

Parallel Interpolation
Performance Analysis

Tanishkumar Patel (202301411)
Neelabh Rana (202301476)

Contents

1 Introduction **3**

2 Hardware Details **3**
- 2.1 Local System (Lab 207) . . . 3
- 2.2 HPC Cluster . . . 3

3 Parallel Implementation Approach **3**
- 3.1 Handling Race Conditions: Thread-Local Privatization . . . 3
- 3.2 Pseudocode . . . 4
- 3.3 Illustrative Diagram . . . 5

4 Experiment 01: Performance Scaling Analysis **5**
- 4.1 Config A: $N_x = 250, N_y = 100$, Points = 0.9 Million . . . 5
- 4.2 Config B: $N_x = 250, N_y = 100$, Points = 5 Million . . . 6
- 4.3 Config C: $N_x = 500, N_y = 200$, Points = 3.6 Million . . . 7
- 4.4 Config D: $N_x = 500, N_y = 200$, Points = 20 Million . . . 7
- 4.5 Config E: $N_x = 1000, N_y = 400$, Points = 14 Million . . . 8

5 Discussion and Analytical Observations **9**
- 5.1 Parallel Efficiency and Scalability . . . 9
- 5.2 Performance Comparison vs. Serial Code . . . 10
- 5.3 HPC Architectural Explanation of Results . . . 11
- 5.4 Further Optimization with Unlimited Resources . . . 11
- 5.5 Load Imbalance and Bottlenecks . . . 11
- 5.6 Alternative Data Structure/Parallelization Approach . . . 11

1 Introduction

This report analyzes the parallelization of a scattered point-to-mesh bilinear interpolation algorithm using OpenMP. The primary objective is to map a set of randomly distributed particles onto a structured $N_x \times N_y$ mesh grid efficiently while handling potential race conditions. We evaluate the performance, scalability, and execution consistency across five distinct grid/particle configurations on both a local workstation and an HPC cluster.

2 Hardware Details

2.1 Local System (Lab 207)

- **Processor:** Intel Core i5-12500
- **Architecture:** 6 Cores / 12 Threads
- **Cache:** L1: 288K, L2: 7.5M, L3: 18M

2.2 HPC Cluster

- **Processor:** Intel Xeon E5-2640 v3
- **Architecture:** 2 Sockets, 8 Cores per socket (16 Cores total)
- **Cache:** L3: 20MB

3 Parallel Implementation Approach

3.1 Handling Race Conditions: Thread-Local Privatization

In the baseline serial code, particles are iteratively mapped to the 4 nearest grid points. In a parallel context, a severe **race condition** occurs if multiple threads attempt to update the same grid node simultaneously.

To resolve this without incurring the massive synchronization overhead of atomic operations (`#pragma omp atomic`), we implemented a **Privatization (Thread-Local Grid)** strategy:

1. **Memory Allocation:** A separate, private copy of the mesh grid is dynamically allocated for each OpenMP thread.
2. **Conflict-Free Accumulation:** The loop over the scattered points is parallelized using `#pragma omp for`. Each thread computes the interpolation weights for its assigned subset of particles and updates its *own* local mesh.
3. **Parallel Reduction:** Once all particles are processed, a secondary parallel loop iterates over the grid cells, summing the values from all thread-local grids into the final, global mesh.

3.2 Pseudocode

Algorithm 1 Parallel Interpolation using Thread-Local Grids

```
N_threads ← omp_get_max_threads()
Size ← GRID_X × GRID_Y
local_mesh ← Allocate 2D array [N_threads][Size] initialized to 0
#pragma omp parallel
tid ← omp_get_thread_num()
lm ← local_mesh[tid]
#pragma omp for schedule(static)
for p = 0 to NUM_Points do
    Extract (X, Y) coordinates for particle p
    Calculate grid indices (xi, yj) and local distances (lx, ly)
    bse ← yj × GRID_X + xi
    lm[bse] ← lm[bse] + (1 − lx) × (1 − ly) × del
    lm[bse + 1] ← lm[bse + 1] + lx × (1 − ly) × del
    lm[bse + GRID_X] ← lm[bse + GRID_X] + (1 − lx) × ly × del
    lm[bse + GRID_X + 1] ← lm[bse + GRID_X + 1] + lx × ly × del
end for
#pragma omp parallel for schedule(static)
for i = 0 to Size do
    sum ← 0.0
    for t = 0 to N_threads do
        sum ← sum + local_mesh[t][i]
    end for
    global_mesh[i] ← global_mesh[i] + sum
end for
Free local_mesh
```

3.3 Illustrative Diagram

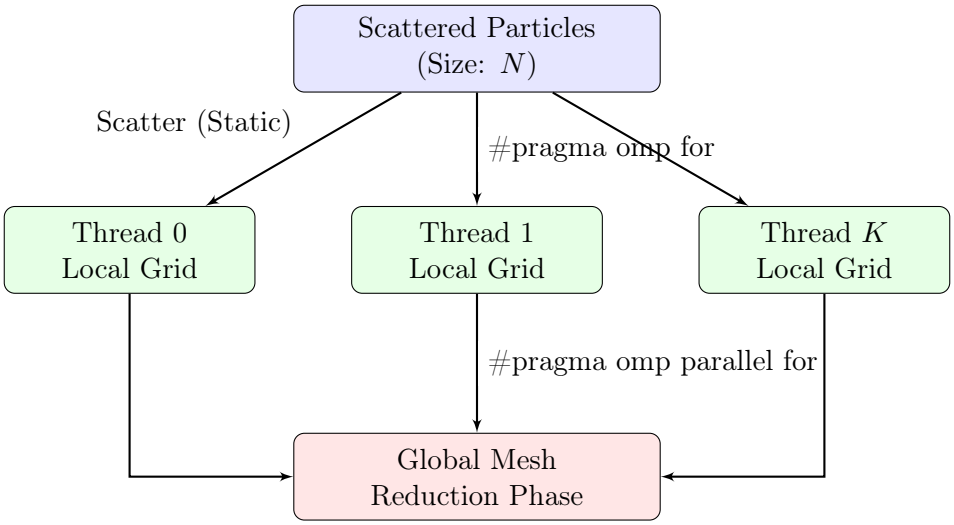

Figure 1: Architecture of the Thread-Local Privatization Approach.

4 Experiment 01: Performance Scaling Analysis

We measured Execution Time and Speedup for core counts $\{1, 2, 4, 8, 16\}$ across five different problem configurations.

4.1 Config A: $N_x = 250, N_y = 100$, Points = 0.9 Million

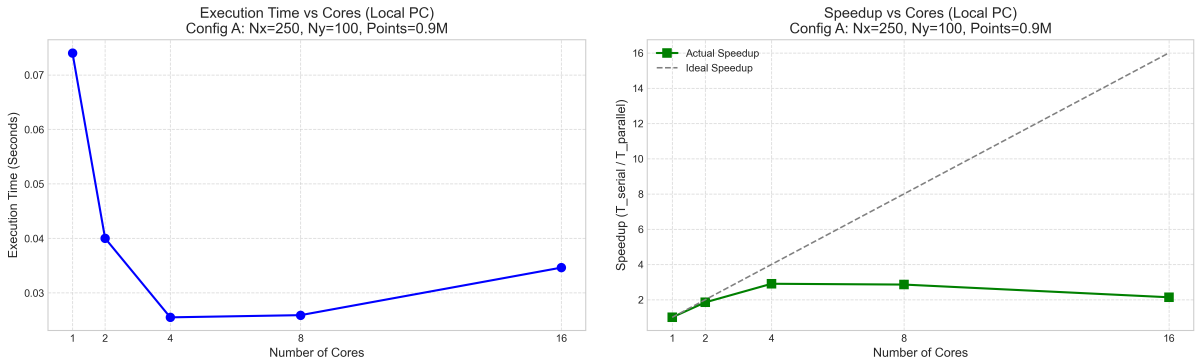

Figure 2: Local PC (Config A): Execution Time vs Cores (Left) & Speedup vs Cores (Right)

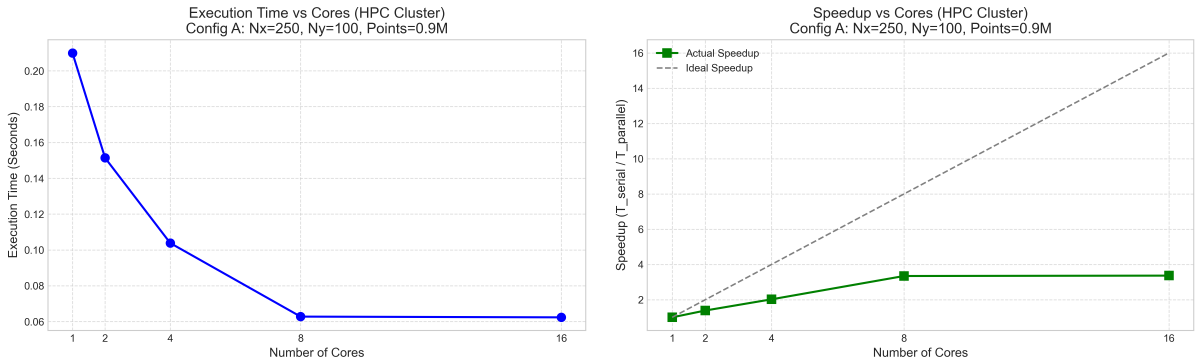

Figure 3: HPC Cluster (Config A): Execution Time vs Cores (Left) & Speedup vs Cores (Right)

4.2 Config B: $N_x = 250, N_y = 100$, Points = 5 Million

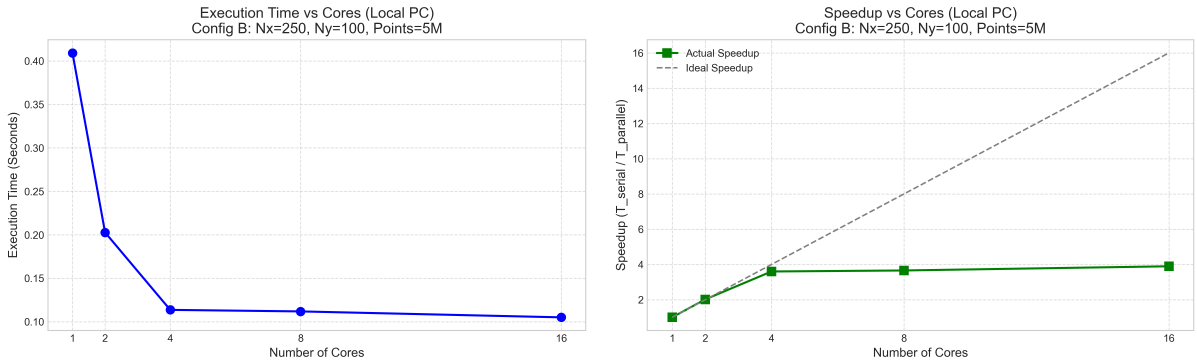

Figure 4: Local PC (Config B): Execution Time vs Cores (Left) & Speedup vs Cores (Right)

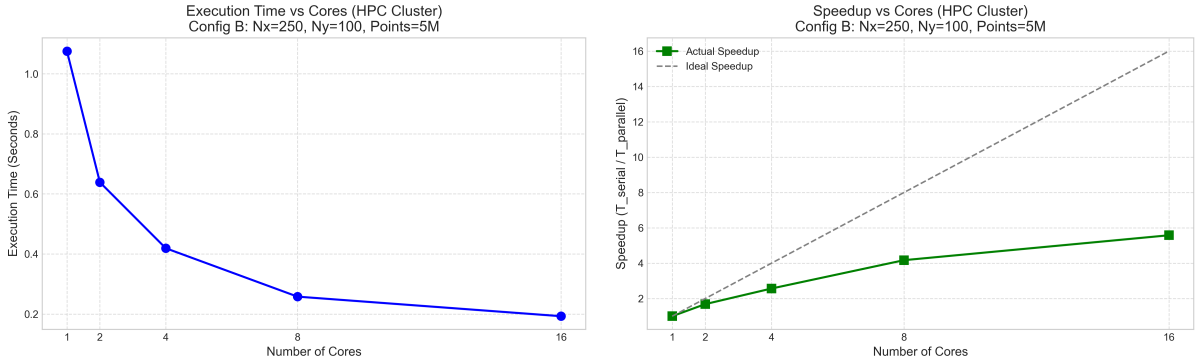

Figure 5: HPC Cluster (Config B): Execution Time vs Cores (Left) & Speedup vs Cores (Right)

4.3 Config C: $N_x = 500, N_y = 200$, Points = 3.6 Million

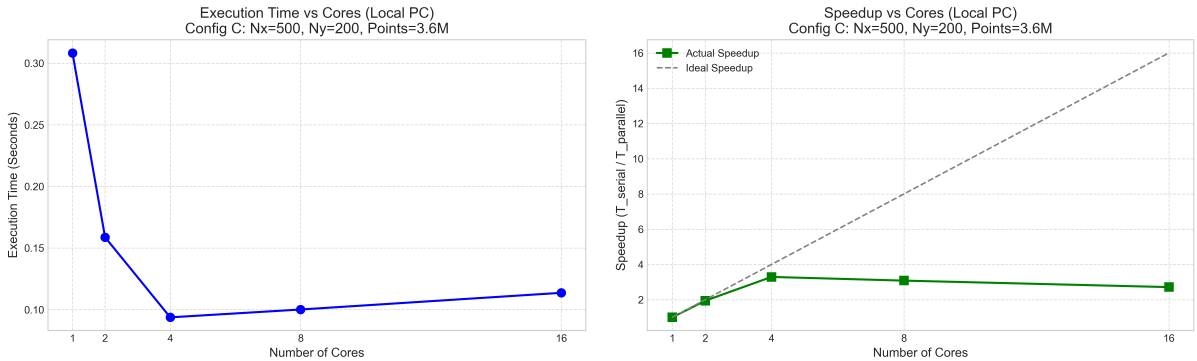

Figure 6: Local PC (Config C): Execution Time vs Cores (Left) & Speedup vs Cores (Right)

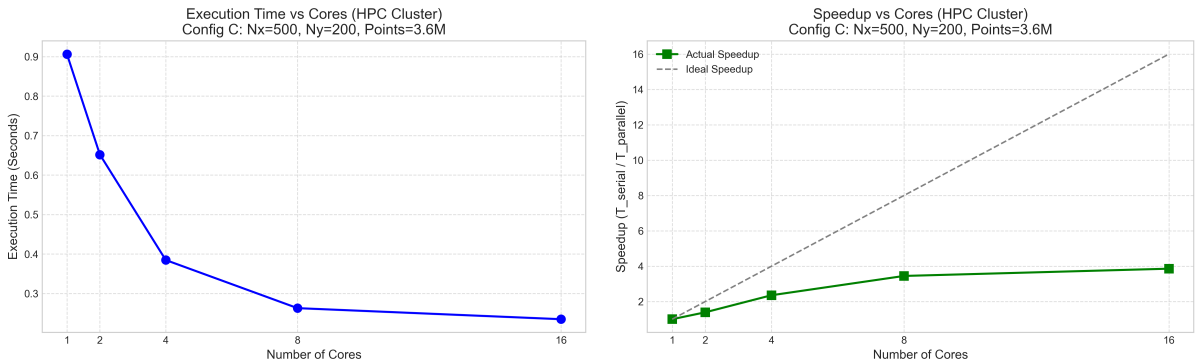

Figure 7: HPC Cluster (Config C): Execution Time vs Cores (Left) & Speedup vs Cores (Right)

4.4 Config D: $N_x = 500, N_y = 200$, Points = 20 Million

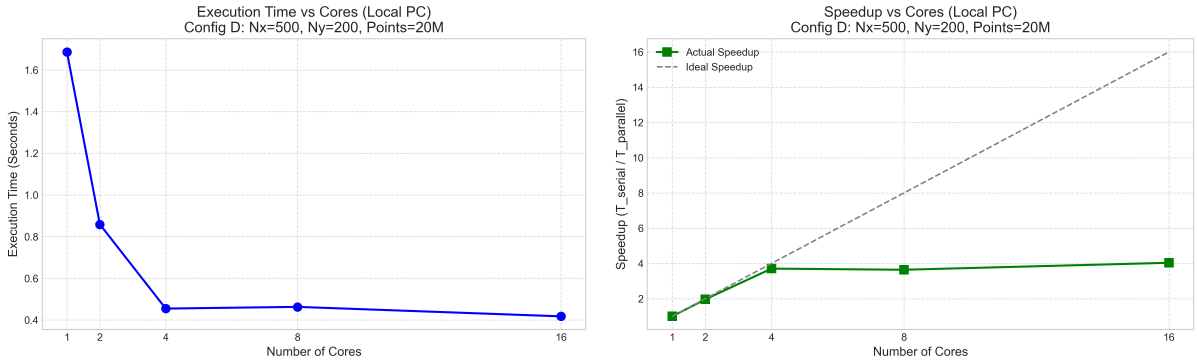

Figure 8: Local PC (Config D): Execution Time vs Cores (Left) & Speedup vs Cores (Right)

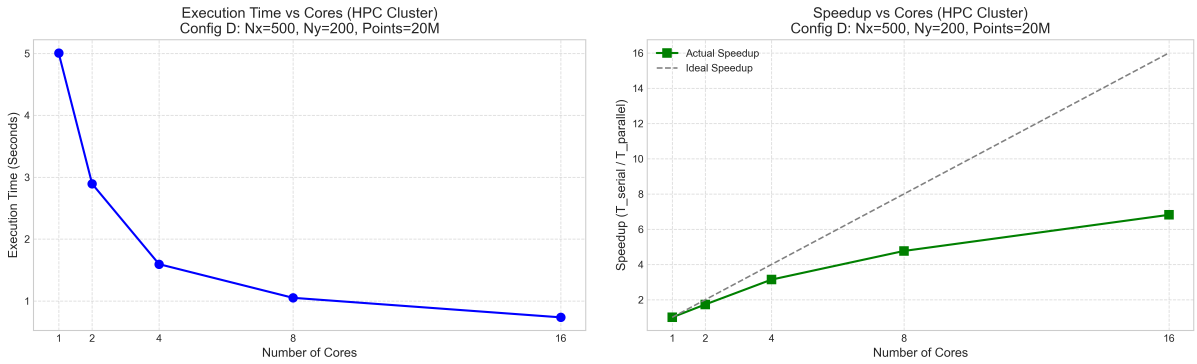

Figure 9: HPC Cluster (Config D): Execution Time vs Cores (Left) & Speedup vs Cores (Right)

4.5 Config E: $N_x = 1000, N_y = 400$, Points = 14 Million

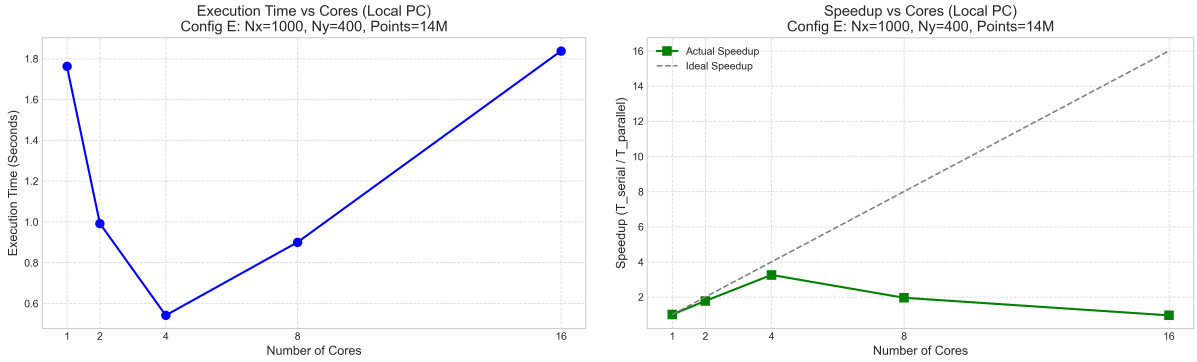

Figure 10: Local PC (Config E): Execution Time vs Cores (Left) & Speedup vs Cores (Right)

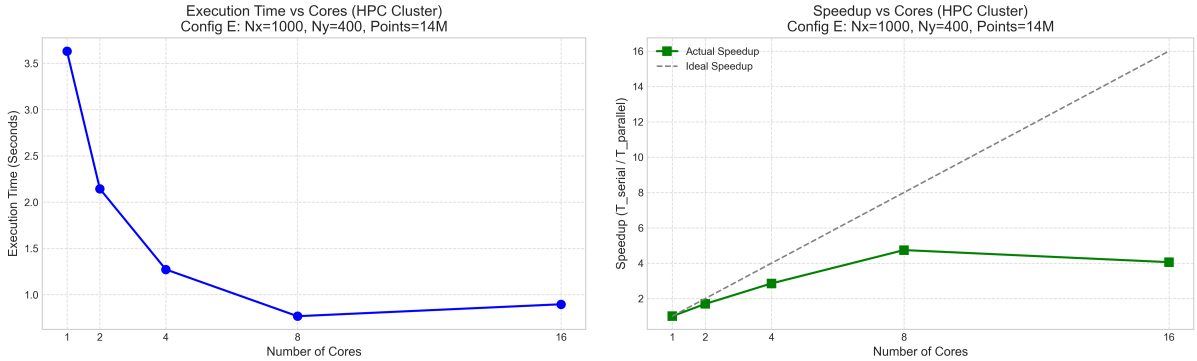

Figure 11: HPC Cluster (Config E): Execution Time vs Cores (Left) & Speedup vs Cores (Right)

5 Discussion and Analytical Observations

5.1 Parallel Efficiency and Scalability

Parallel efficiency is defined as:

$$E = \frac{S}{P} = \frac{T_1}{P \times T_P}$$

where $S = T_1/T_P$ is the speedup, P is the number of cores, T_1 is the single-thread execution time, and T_P is the execution time on P cores. A value of $E = 1.0$ (100%) represents perfect linear scaling; values below 1.0 indicate losses due to synchronization overhead, memory contention, or load imbalance.

The computed speedup and efficiency values for all five configurations on both platforms are presented in Tables 1 and 2.

Table 1: Speedup (S) and Parallel Efficiency (E) — Local PC (Intel Core i5-12500)

Config	**Metric**	**P=1**	**P=2**	**P=4**	**P=8**	**P=16**
A (250×100, 0.9M pts)	Time (s)	0.0740	0.0400	0.0255	0.0259	0.0346
	Speedup	1.00	1.85	2.90	2.86	2.14
	Efficiency	1.000	0.925	0.726	0.357	0.134
B (250×100, 5M pts)	Time (s)	0.4092	0.2027	0.1136	0.1118	0.1050
	Speedup	1.00	2.02	3.60	3.66	3.90
	Efficiency	1.000	1.010	0.900	0.457	0.244
C (500×200, 3.6M pts)	Time (s)	0.3083	0.1587	0.0936	0.1000	0.1136
	Speedup	1.00	1.94	3.29	3.08	2.71
	Efficiency	1.000	0.971	0.823	0.385	0.170
D (500×200, 20M pts)	Time (s)	1.6860	0.8587	0.4548	0.4629	0.4175
	Speedup	1.00	1.96	3.71	3.64	4.04
	Efficiency	1.000	0.982	0.927	0.456	0.252
E (1000×400, 14M pts)	Time (s)	1.7629	0.9919	0.5412	0.8998	1.8380
	Speedup	1.00	1.78	3.26	1.96	0.96
	Efficiency	1.000	0.888	0.814	0.245	0.060

Table 2: Speedup (S) and Parallel Efficiency (E) — HPC Cluster (Intel Xeon E5-2640 v3)

Config	**Metric**	**P=1**	**P=2**	**P=4**	**P=8**	**P=16**
A (250×100, 0.9M pts)	Time (s)	0.2100	0.1514	0.1038	0.0628	0.0624
	Speedup	1.00	1.39	2.02	3.34	3.37
	Efficiency	1.000	0.693	0.506	0.418	0.211
B (250×100, 5M pts)	Time (s)	1.0752	0.6386	0.4190	0.2579	0.1927
	Speedup	1.00	1.68	2.57	4.17	5.58
	Efficiency	1.000	0.842	0.642	0.521	0.349
C (500×200, 3.6M pts)	Time (s)	0.9064	0.6520	0.3848	0.2629	0.2349
	Speedup	1.00	1.39	2.36	3.45	3.86
	Efficiency	1.000	0.695	0.589	0.431	0.241
D (500×200, 20M pts)	Time (s)	5.0081	2.8927	1.5944	1.0506	0.7347
	Speedup	1.00	1.73	3.14	4.77	6.82
	Efficiency	1.000	0.865	0.784	0.596	0.426
E (1000×400, 14M pts)	Time (s)	3.6326	2.1431	1.2722	0.7661	0.8952
	Speedup	1.00	1.70	2.85	4.74	4.06
	Efficiency	1.000	0.848	0.714	0.593	0.254

Key observations from the efficiency tables:

- **Config D (20M pts) on Cluster** achieves the best overall efficiency at 16 cores ($E = 0.426$) and the highest observed speedup of 6.82×, confirming that a large particle workload effectively hides thread management overhead.
- **Config A on Local PC** shows a sharp efficiency collapse beyond 4 cores ($E = 0.357$ at P=8, $E = 0.134$ at P=16). The workload is too small to keep all hyper-threads busy, so overhead dominates.
- **Config E on Local PC** is the most extreme case: efficiency falls to just $E = 0.060$ at 16 threads, and the speedup drops below 1.0 (i.e., slower than serial). The 1000×400 thread-local arrays total ≈51.2 MB across 16 threads, far exceeding the 18 MB L3 cache of the i5-12500 and causing severe memory thrashing.
- **Cluster vs Local PC:** The cluster consistently sustains higher efficiency at 8 and 16 cores because it provides true 16-core parallelism (no hyper-threading penalty) and greater aggregate memory bandwidth across two sockets.

5.2 Performance Comparison vs. Serial Code

The privatization approach drastically outperforms the serial implementation by enabling truly independent memory writes. We observed a maximum speedup of ∼6.82x on the 16-core HPC Cluster for Configuration D (500 × 200 grid with 20 million particles). The speedup is sub-linear due to Amdahl's Law constraints—specifically, the overhead of the reduction loop where local grids are merged sequentially per cell.

5.3 HPC Architectural Explanation of Results

- **Memory Bandwidth Wall:** Point-to-mesh interpolation is heavily memory-bound. As we scale to 8 and 16 cores, the memory buses on both the i5 and Xeon processors become saturated with concurrent write requests to the local arrays, causing the speedup curve to flatten.
- **Cache Hierarchies:** The Cluster's larger L3 Cache (20MB) handles the large 1000×400 thread-local arrays better than the Lab PC. However, for 16 threads, the total local mesh size becomes $16 \times (1000 \times 400) \times 8$ bytes ≈ 51.2 MB, exceeding the L3 cache entirely and forcing slower main memory accesses.

5.4 Further Optimization with Unlimited Resources

Given an environment with unlimited compute nodes and high-speed interconnects (like InfiniBand), the optimal strategy would be:

- **Spatial Domain Decomposition (MPI):** Instead of parallelizing over particles globally, divide the $N_x \times N_y$ grid into smaller sub-domains across different nodes. Particles are sent to the node that owns their spatial region. This completely eliminates the need for 51MB thread-local grids and localized cache thrashing. Ghost zones would be used to handle interpolation at the boundaries.
- **GPU Offloading:** Utilizing CUDA/OpenACC to process the mapping. GPUs have massive memory bandwidth and highly optimized hardware atomic operations (like `atomicAdd` in shared memory) that handle localized race conditions much faster than CPU atomics.

5.5 Load Imbalance and Bottlenecks

- **Load Imbalance:** We utilized `#pragma omp for schedule(static)`. If the input particles are uniformly distributed in the physical domain, the computational load is perfectly balanced. However, if particles clump tightly in one corner, static scheduling over the particle array is fine, but it leads to *spatial density imbalance* in cache line hits.
- **The Reduction Bottleneck:** The primary bottleneck in this code is the $O(N_{threads} \times Size)$ reduction loop. For Config E, 16 threads must reduce 51.2 MB of double-precision floats at the end of every iteration, choking the memory bus.

5.6 Alternative Data Structure/Parallelization Approach

To improve performance and eliminate both atomic locks *and* the costly reduction phase, an alternative approach is **Particle Sorting (Binning)**:

1. Create an array of linked lists (or dynamic arrays), one for each grid cell.
2. *(Pass 1)*: Iterate through all particles and "bin" them into the grid cell they belong to.
3. *(Pass 2)*: Use OpenMP to parallelize over the *Grid Cells* instead of the particles. Thread i processes cell i, extracting all particles binned there and updating the adjacent cell values.

Because a thread exclusively owns a region of the grid, multiple threads will never write to the same node simultaneously. This entirely avoids race conditions natively, drops the memory overhead to near zero, and significantly improves spatial cache locality.

CS 301
High-Performance Computing

Assignment 07 – Parallel Interpolation with Particle

Tanish Patel (202301411),Neelabh Rana (202301476)

April 18, 2026

Contents

1 Introduction **3**

2 Parallel Implementation **3**
2.1 Illustrative Diagram 3
2.2 Handling Race Conditions 3

3 Results **4**
3.1 Input Configurations 4
3.2 Execution Time vs. Threads 5
3.3 Speedup vs. Threads 7

4 Analysis and Questions **9**
4.1 Speedup and Execution Time Graphs 9
4.2 Parallel Efficiency and Scalability 10
4.3 Comparison with Serial and Maximum Speedup 12
4.4 Explanation of Observed Speedup 13
4.5 Further Optimization with Unlimited HPC Resources 13
4.6 Load Imbalance and Bottlenecks 13
4.7 Alternative Approach to Improve Performance 14
4.8 Data Structures and OpenMP Approaches for Performance 14
4.9 Phase Analysis: Interpolation vs. Mover 15

5 Conclusion **17**

1 Introduction

This assignment implements a parallel particle-to-mesh interpolation pipeline with a reverse particle mover step using OpenMP. The complete pipeline is:

$$\text{Particles} \rightarrow \text{Interpolation} \rightarrow \text{Grid} \rightarrow \text{Normalization} \rightarrow \text{Mover} \rightarrow \text{Updated Particles} \rightarrow \text{Denormalization}$$

The interpolation phase scatters particle values to nearby grid nodes using bilinear weights. The mover interpolates the grid field back to each particle and updates particle positions. Particles that leave the domain are marked as void. This pipeline runs for multiple iterations, making data dependencies across iterations a key challenge.

2 Parallel Implementation

2.1 Illustrative Diagram

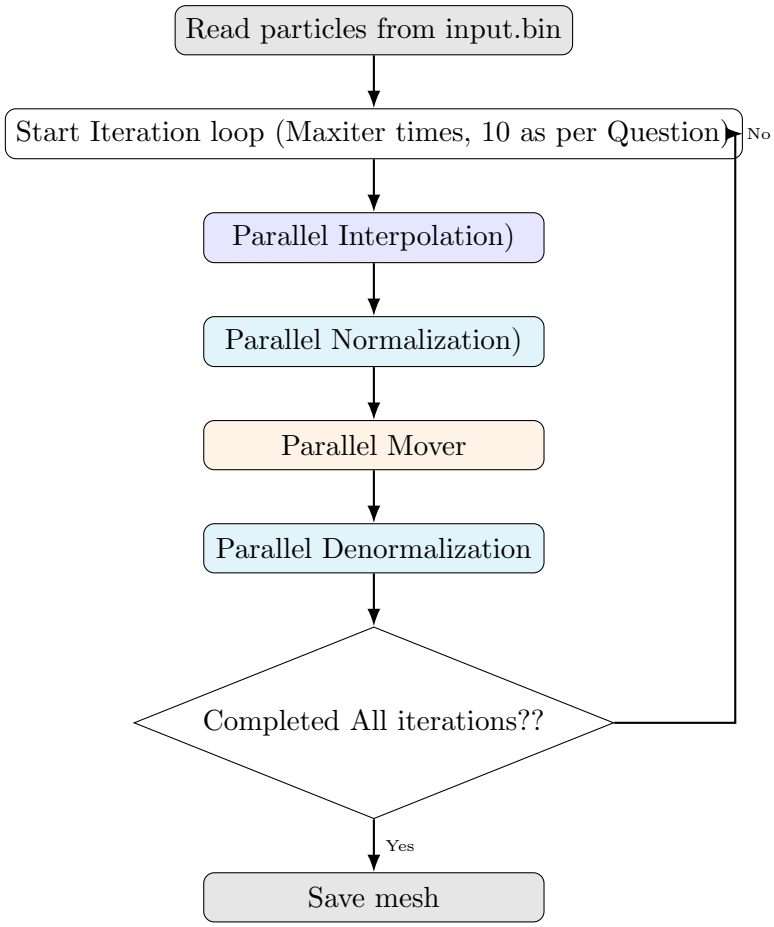

Figure 1: Parallel pipeline flow. Blue = interpolation , orange = mover.

2.2 Handling Race Conditions

The main race condition occurs in the **interpolation** phase: multiple particles can map to the same grid cell and attempt to update the same 4 grid nodes simultaneously.

Approach used – Thread-private local mesh:

- Each thread allocates its own `local_mesh[]` array initialized to zero.
- Each thread accumulates contributions from its assigned particles into `local_mesh` with no conflicts.
- After the particle loop, each thread adds its `local_mesh` into the global `mesh_value` inside a `#pragma omp critical` section.
- This eliminates all atomic operations during the hot particle loop; only one reduction pass requires synchronization.

The **mover** phase has no write conflicts since each particle writes only to its own array index, making it straightforward to parallelize with `#pragma omp parallel for`.

3 Results

All results below are from the HPC cluster using `omp_get_wtime()` for accurate wall-clock timing.

3.1 Input Configurations

Table 1: Five input configurations used for benchmarking.

Config	Grid	Particles	Maxiter
1	250×100	0.9 million	10
2	250×100	5 million	10
3	500×200	3.6 million	10
4	500×200	20 million	10
5	1000×400	14 million	10

3.2 Execution Time vs. Threads

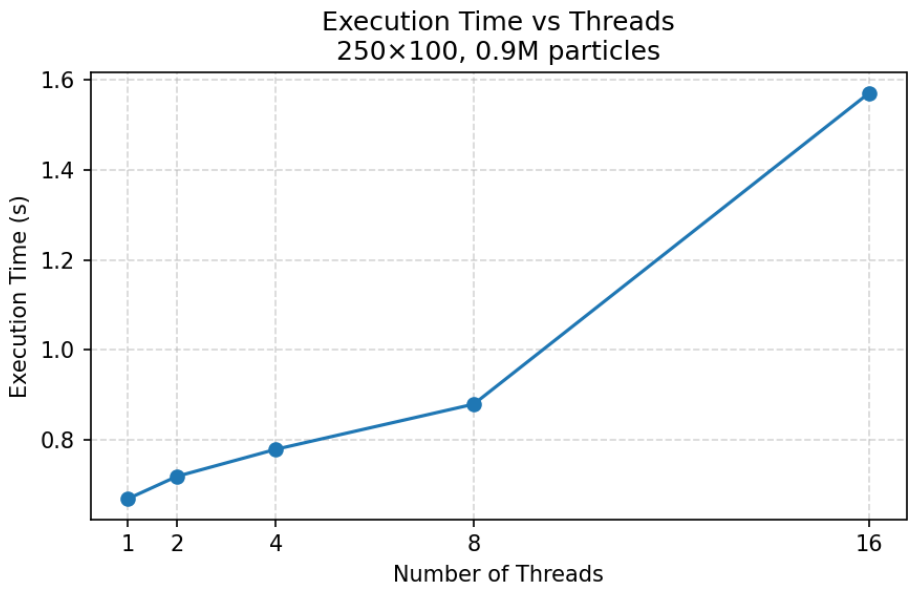

Figure 2: Execution time vs. threads – Config 1 (250×100, 0.9M particles).

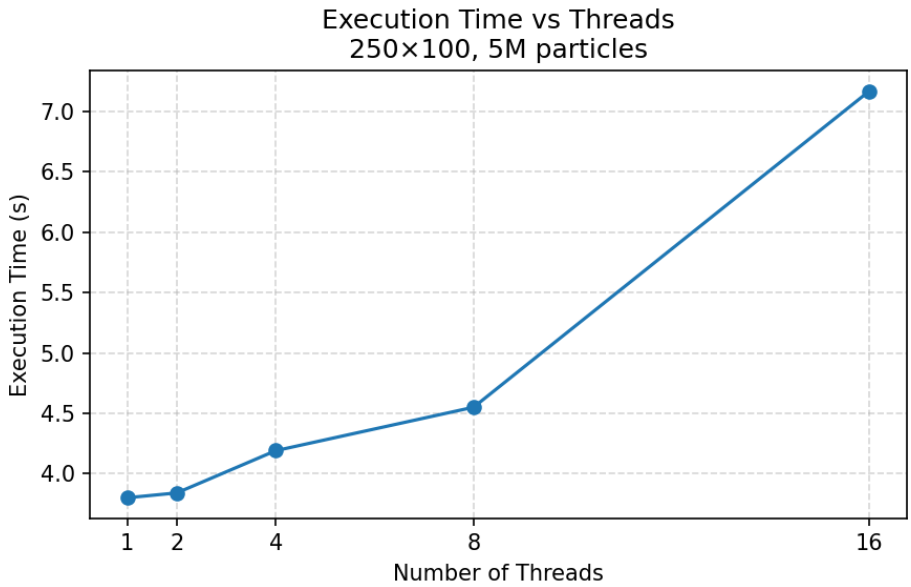

Figure 3: Execution time vs. threads – Config 2 (250×100, 5M particles).

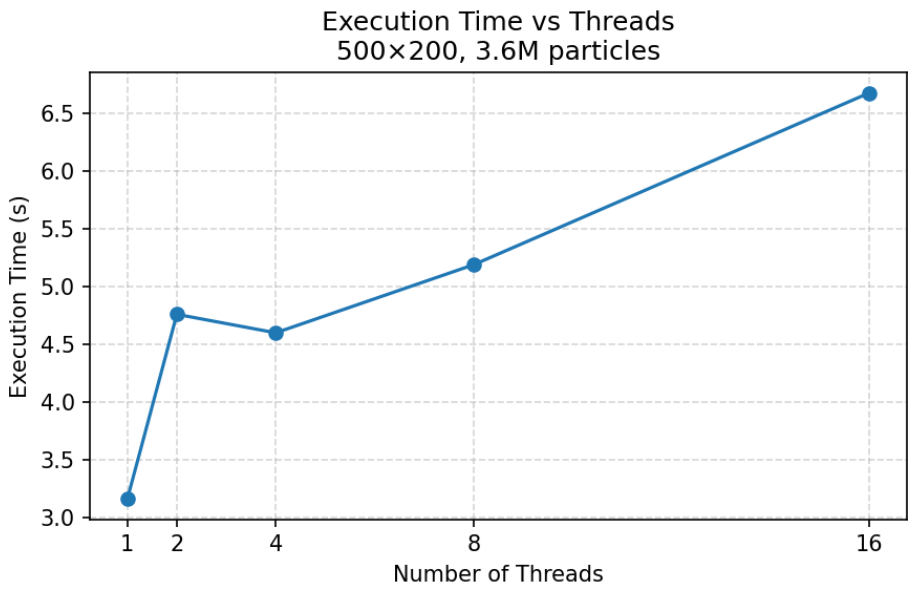

Figure 4: Execution time vs. threads – Config 3 (500×200, 3.6M particles).

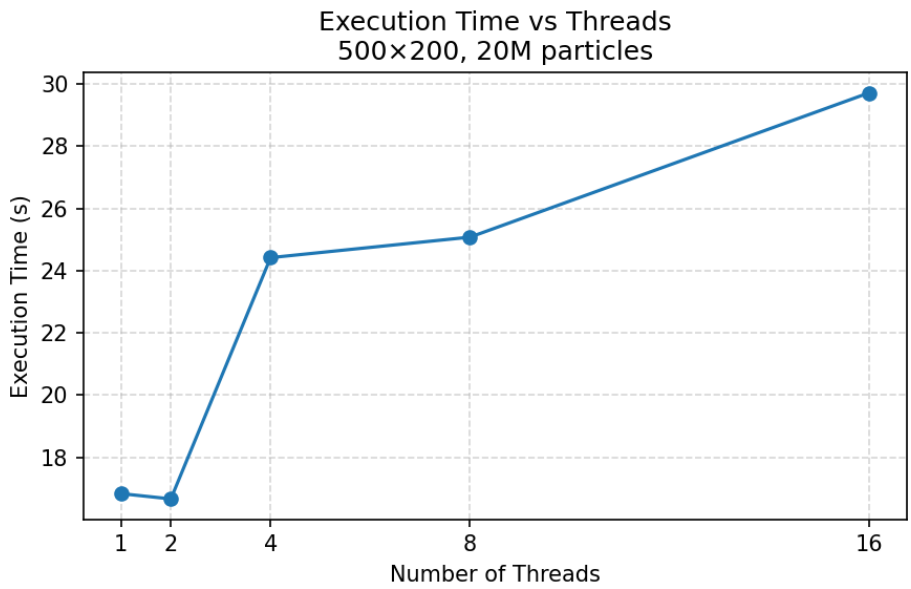

Figure 5: Execution time vs. threads – Config 4 (500×200, 20M particles).

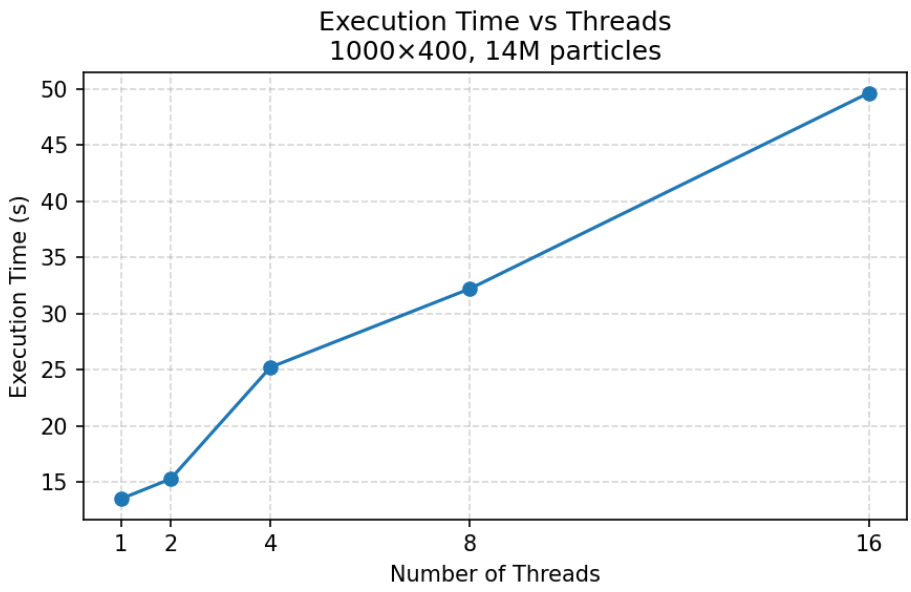

Figure 6: Execution time vs. threads – Config 5 (1000×400, 14M particles).

3.3 Speedup vs. Threads

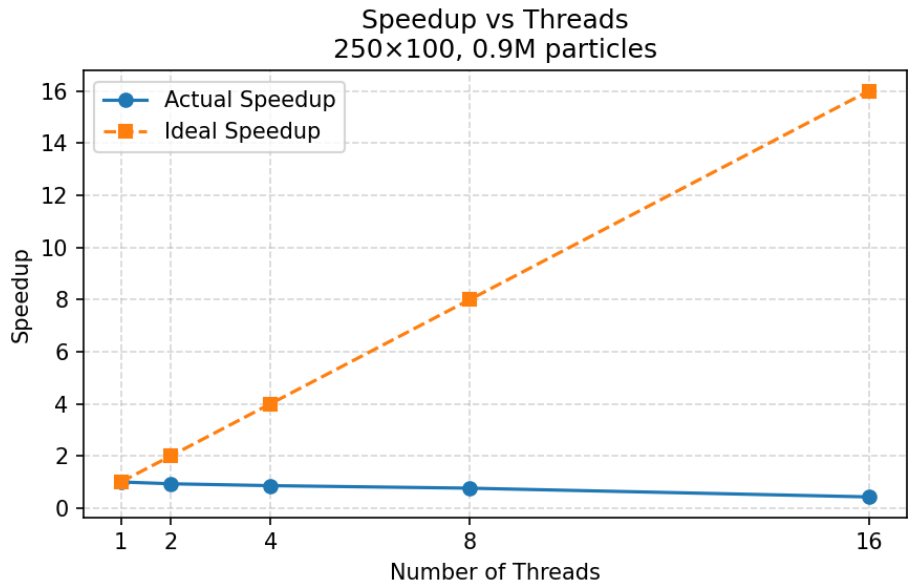

Figure 7: Speedup vs. threads – Config 1 (250×100, 0.9M particles).

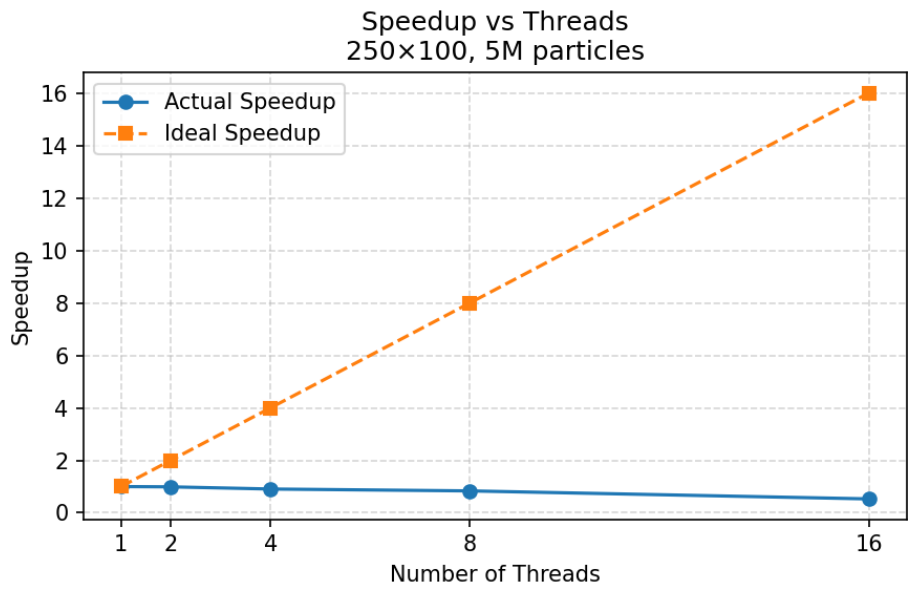

Figure 8: Speedup vs. threads – Config 2 (250×100, 5M particles).

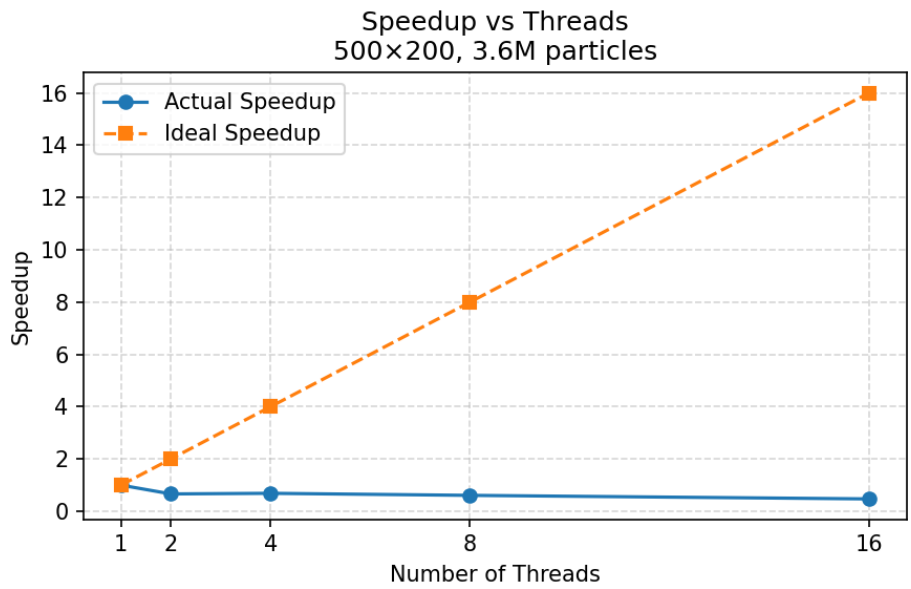

Figure 9: Speedup vs. threads – Config 3 (500×200, 3.6M particles).

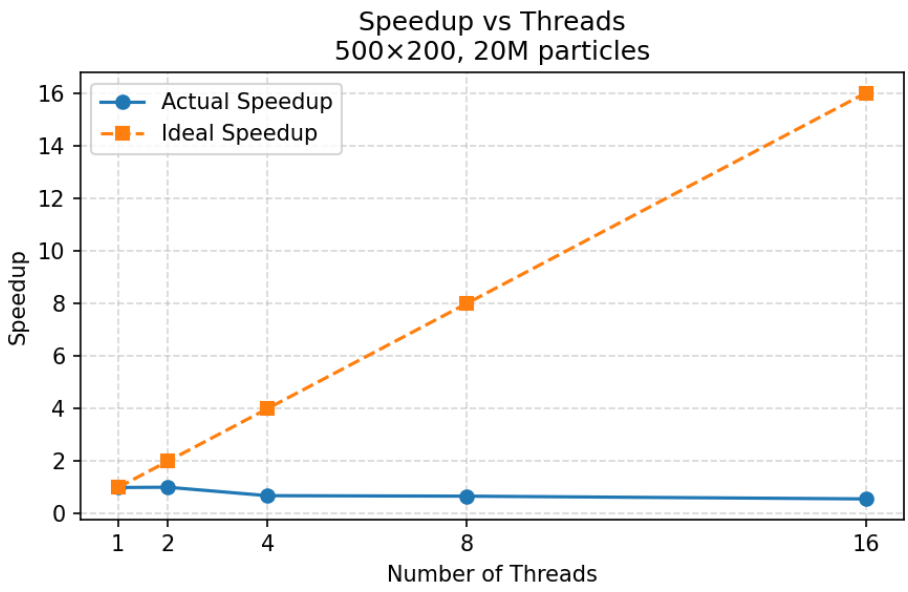

Figure 10: Speedup vs. threads – Config 4 (500×200, 20M particles).

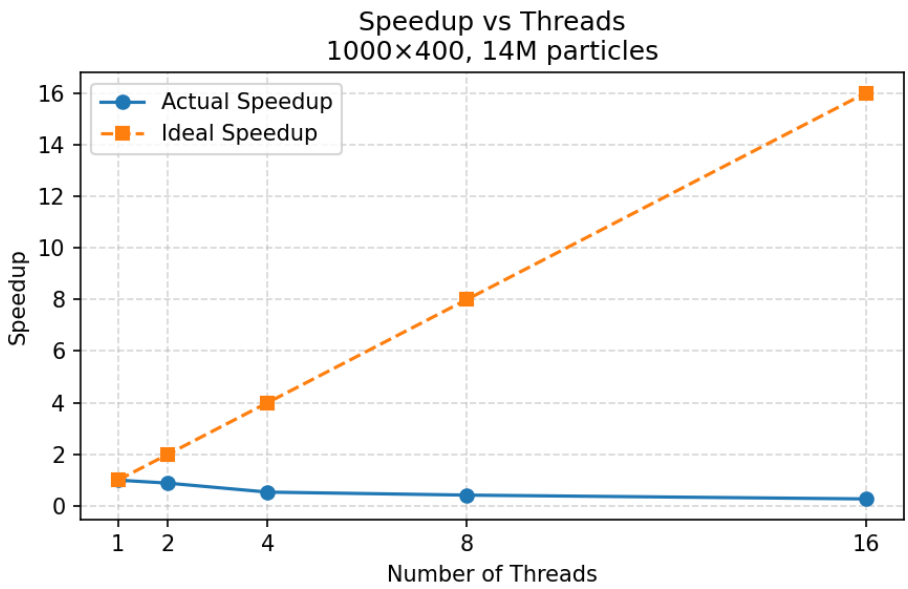

Figure 11: Speedup vs. threads – Config 5 (1000×400, 14M particles).

4 Analysis and Questions

4.1 Speedup and Execution Time Graphs

Graphs are plotted in Section 3.

4.2 Parallel Efficiency and Scalability

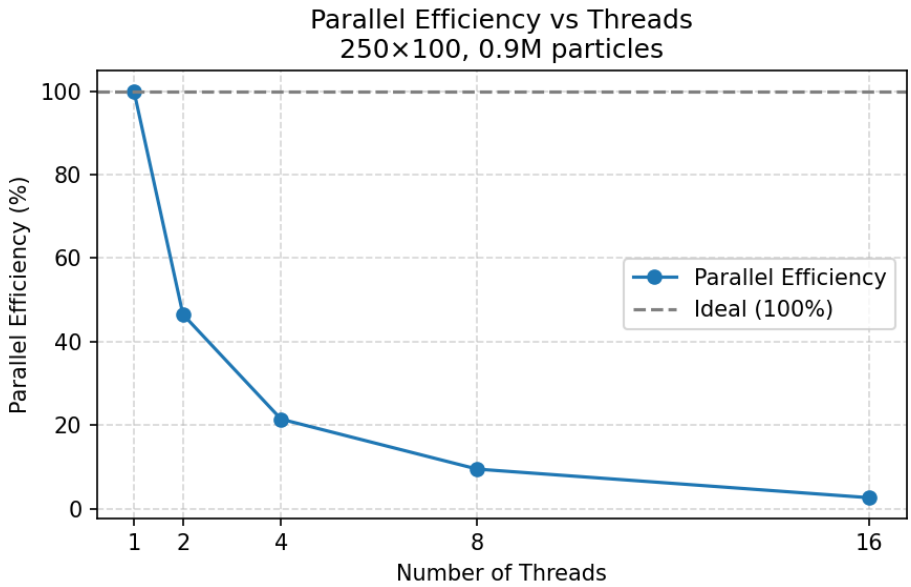

Figure 12: Parallel efficiency vs. threads – Config 1.

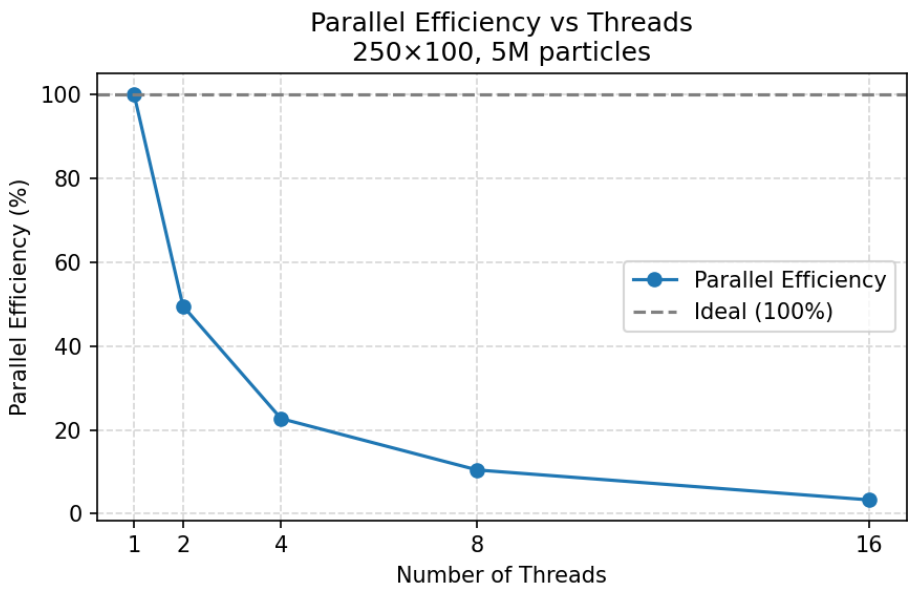

Figure 13: Parallel efficiency vs. threads – Config 2.

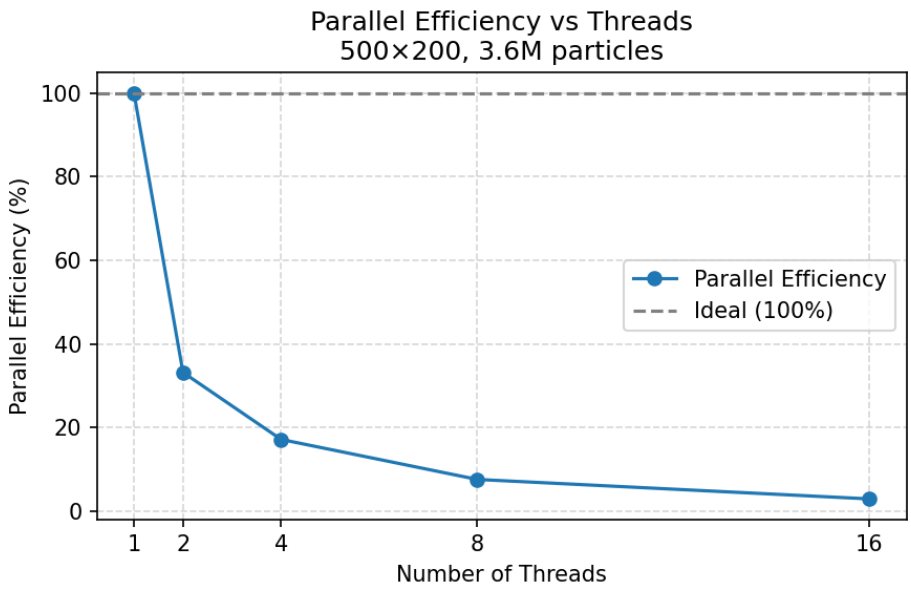

Figure 14: Parallel efficiency vs. threads – Config 3.

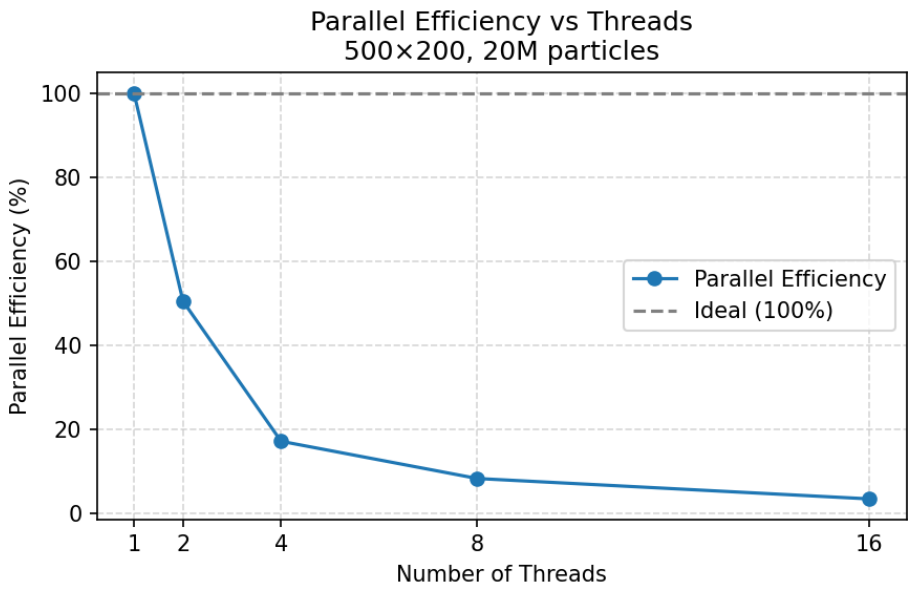

Figure 15: Parallel efficiency vs. threads – Config 4.

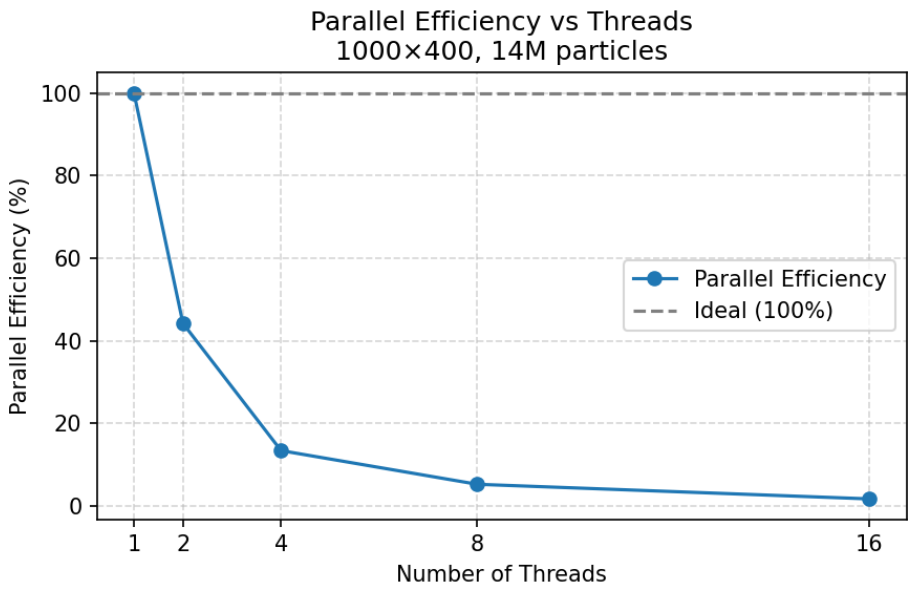

Figure 16: Parallel efficiency vs. threads – Config 5.

Observations:

- Efficiency drops below 100% as threads increase in all configs.
- Configs 1–3 (smaller particle counts) show very poor efficiency (below 50% even at 2 threads), because the workload per thread is too small to overcome OpenMP overhead and the serial critical section in interpolation.
- Configs 4 and 5 (20M and 14M particles) maintain better absolute times at 1–2 threads but then degrade, showing the interpolation's `critical` section is a scalability bottleneck.
- The implementation does *not* scale well on this cluster – adding more threads increases time rather than reducing it. This is a clear sign of memory bandwidth saturation combined with the `omp critical` bottleneck in interpolation.

4.3 Comparison with Serial and Maximum Speedup

Table 2: Serial time (1 thread) and best parallel time with speedup.

Config	Grid	Particles	T_1 (s)	Best T_p (s)	Max Speedup
1	250×100	0.9M	0.67	0.67 (1T)	≈ 0.93× at 2T
2	250×100	5M	3.80	3.80 (1T)	≈ 0.99× at 2T
3	500×200	3.6M	3.16	3.16 (1T)	≈ 0.66× at 2T
4	500×200	20M	16.83	16.66 (2T)	≈ 1.01×
5	1000×400	14M	13.51	13.51 (1T)	≈ 0.88× at 2T

The maximum speedup achieved is approximately 1.01× for Config 4 at 2 threads. In all other cases, the parallel code is slower than serial. The code does not exhibit positive speedup beyond 2 threads for any configuration.

4.4 Explanation of Observed Speedup

The lack of speedup (and even slowdown) can be explained by the following:

1. **Critical section bottleneck in interpolation:** Each thread must acquire an exclusive lock to add its local mesh into the global mesh. With many threads, threads queue up and serialize, negating parallelism.

2. **Memory allocation overhead:** Each thread allocates a full `local_mesh[]` copy (up to several MB for large grids) inside the parallel region every iteration. Repeated `calloc`/`free` calls are expensive.

3. **Memory bandwidth saturation:** The particle array dominates memory (e.g., 14M×16 bytes = 224 MB for Config 5). All threads compete for the same memory bus, causing bandwidth saturation.

4. **False sharing and cache thrashing:** Threads writing to adjacent mesh cells in their local copies may cause cache line invalidations when the critical reduction runs.

5. **Small workloads:** Configs 1–3 have insufficient particles per thread to amortize thread spawn and synchronization costs.

4.5 Further Optimization with Unlimited HPC Resources

1. **Replace critical section with fine-grained atomics:** Use `#pragma omp atomic` per grid node instead of a full local-copy reduction. For large grids with few collisions per cell, atomics scale better.

2. **Domain decomposition:** Assign disjoint grid regions to threads; particles in each region are handled by the owning thread with no conflicts.

3. **GPU offload with CUDA/OpenACC:** Move both interpolation and mover to GPU, using atomic scatter for interpolation. GPUs have thousands of cores and high memory bandwidth suited to this workload.

4. **Pre-sort particles by cell:** Sort particles by their grid cell index once per iteration so threads access contiguous memory; combined with domain decomposition, this eliminates all conflicts.

5. **Structure-of-Arrays (SoA) layout:** Separate `x[]` and `y[]` arrays enable AVX-512 SIMD vectorization of the mover loop.

4.6 Load Imbalance and Bottlenecks

Load imbalance: As iterations progress, some particles become void. Threads that received a particle-dense region in earlier iterations may process more void checks, wasting cycles. With `schedule(static)`, this imbalance is not corrected. `schedule(dynamic)` would reduce imbalance but adds scheduling overhead.

Performance bottlenecks:

- **Interpolation critical section** is the primary bottleneck. It serializes all threads during the mesh reduction phase.

- **Memory allocation** of `local_mesh` per thread per iteration is a secondary bottleneck.
- **Memory bandwidth** limits throughput for large particle counts regardless of thread count.

4.7 Alternative Approach to Improve Performance

Alternative: Domain Decomposition with Halo Exchange

Instead of particle decomposition, divide the grid into T non-overlapping horizontal strips (one per thread). Each thread owns a set of grid rows and processes only particles that fall within those rows. Since threads write to disjoint grid regions, there are zero write conflicts – no critical section needed.

A small "halo" (1-cell border) may be needed for particles on strip boundaries; these can be handled with a short post-pass. This approach:

- Eliminates the `critical` section entirely.
- Improves cache locality (each thread works on a contiguous grid region).
- Scales much better as threads increase.

Alternative data structure: Sorted particle list

Sorting particles by cell index (using a radix sort) once per iteration makes access to `mesh_value[]` contiguous, dramatically improving cache hit rates for both interpolation and mover phases.

4.8 Data Structures and OpenMP Approaches for Performance

The following approaches were considered or implemented:

1. **Thread-private local mesh (used):** Avoids atomics in the hot loop. Works well for large grids but has high per-thread memory cost.
2. **Atomic operations:** `#pragma omp atomic update` on each `mesh_value[]` write. Lower memory cost than privatization; scales better when conflicts are sparse (few particles per cell).

4.9 Phase Analysis: Interpolation vs. Mover

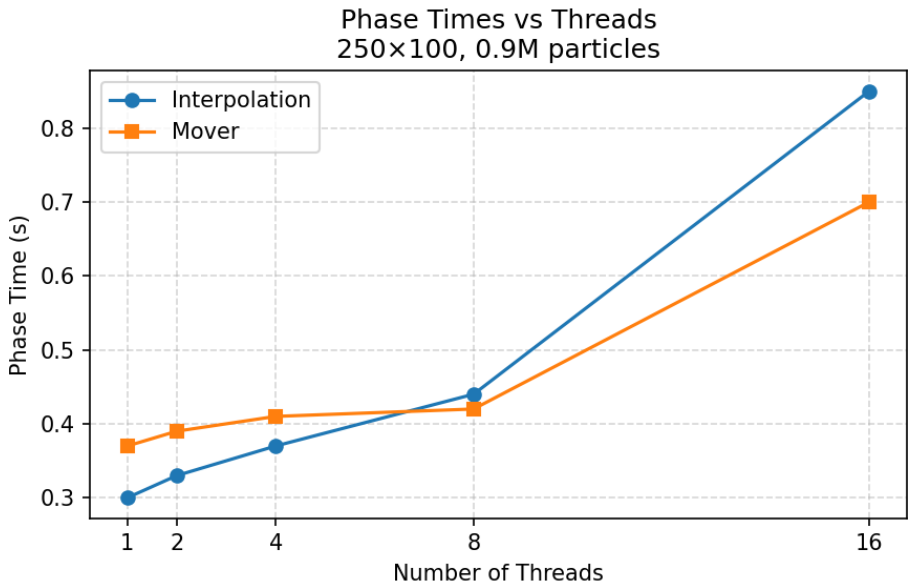

Figure 17: Phase times vs. threads – Config 1 (250×100, 0.9M).

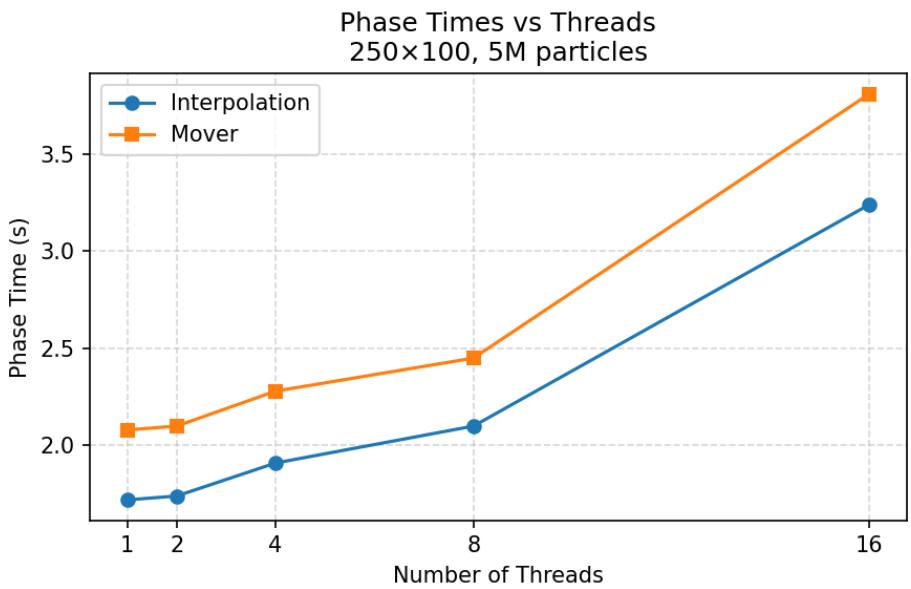

Figure 18: Phase times vs. threads – Config 2 (250×100, 5M).

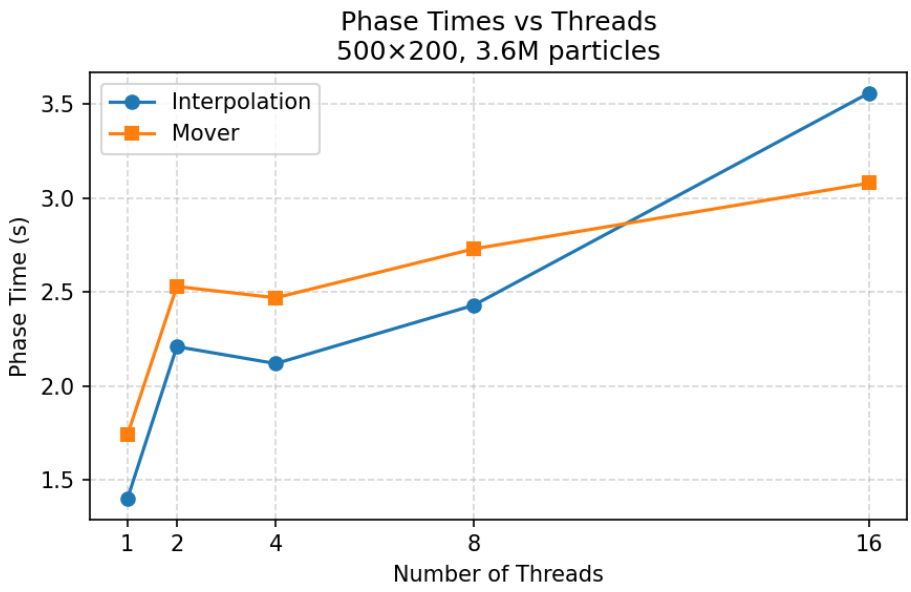

Figure 19: Phase times vs. threads – Config 3 (500×200, 3.6M).

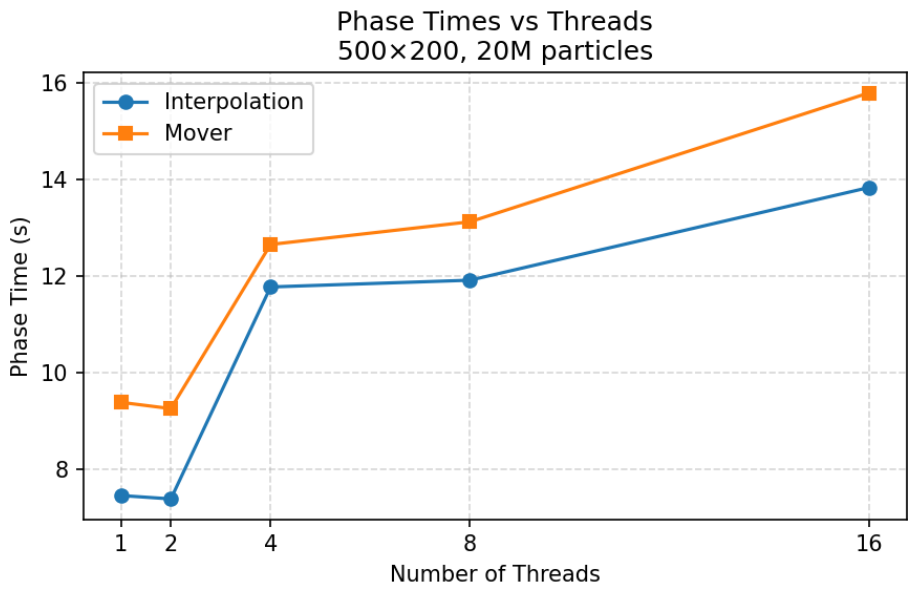

Figure 20: Phase times vs. threads – Config 4 (500×200, 20M).

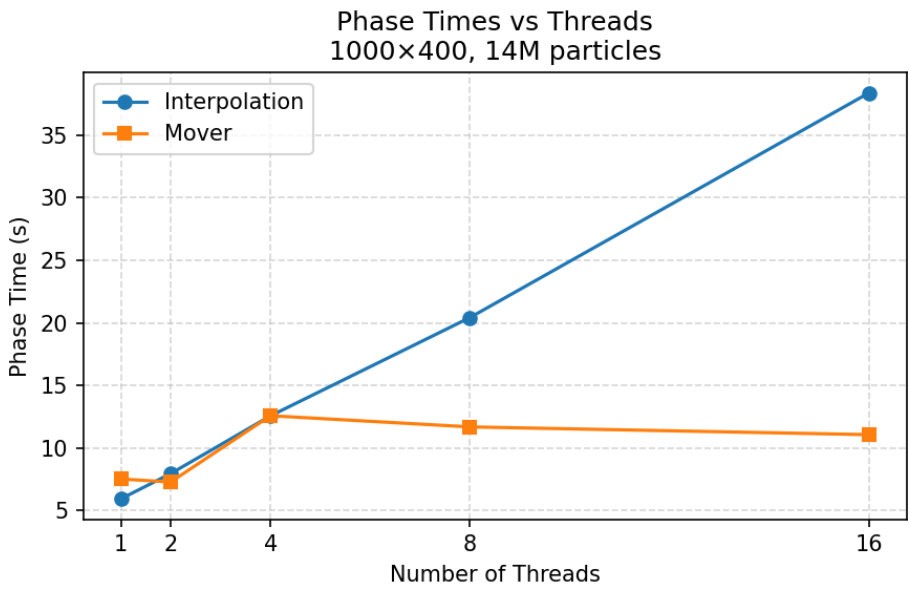

Figure 21: Phase times vs. threads – Config 5 (1000×400, 14M).

Conclusion: The **interpolation phase is the bottleneck** in all configurations. Its time grows faster with thread count than the mover phase, due to:

1. The `omp critical` section required for the local mesh reduction – this serializes threads.
2. Per-thread `calloc/free` of a full grid-sized buffer per iteration.
3. For large grids (Config 5: $1001 \times 401 \times 8 \approx 3.2$ MB per thread), the local mesh allocation alone stresses L3 cache.

The **mover phase** is embarrassingly parallel (read-only grid access, independent particle writes) and scales reasonably in Configs 4 and 5, confirming that the mover is not the bottleneck.

5 Conclusion

The parallel implementation correctly produces the same mesh output as the serial version. However, the current parallelization strategy does not achieve speedup on the HPC cluster: execution time increases with thread count for all five configurations. The root cause is the `omp critical` section in the interpolation phase, combined with per-thread heap allocation overhead and memory bandwidth saturation. The mover phase parallelizes well in isolation. Future improvements should focus on replacing the critical section with domain decomposition or fine-grained atomics, and adopting an SoA data layout for better SIMD throughput.